



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 10. Кейсы
(/teach/control/stream/view/id/931478155)

PErr10.6 Генератор постов для блога в Telegram. Часть 2 (new)

Предыдущий урок

(/pl/teach/control/lesson/view?id=341528145)

Следующий урок

(/pl/teach/control/lesson/view?id=341528207)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Что мы умеем

- ✓ Собираем полноценный workflow в n8n: Webhook → AI → Telegram.
- ✓ Подключаем OpenAI для генерации текста и изображений.
- ✓ Настраиваем HTTP-запросы, передачу binary-файлов и автопостинг в Telegram.
- ✓ Управляем токенами, дорабатываем промпты и обрабатываем ошибки.
- ✓ Разворачиваем систему на сервере через Docker и защищаем её через Authorization header.

Действия

Результат дня

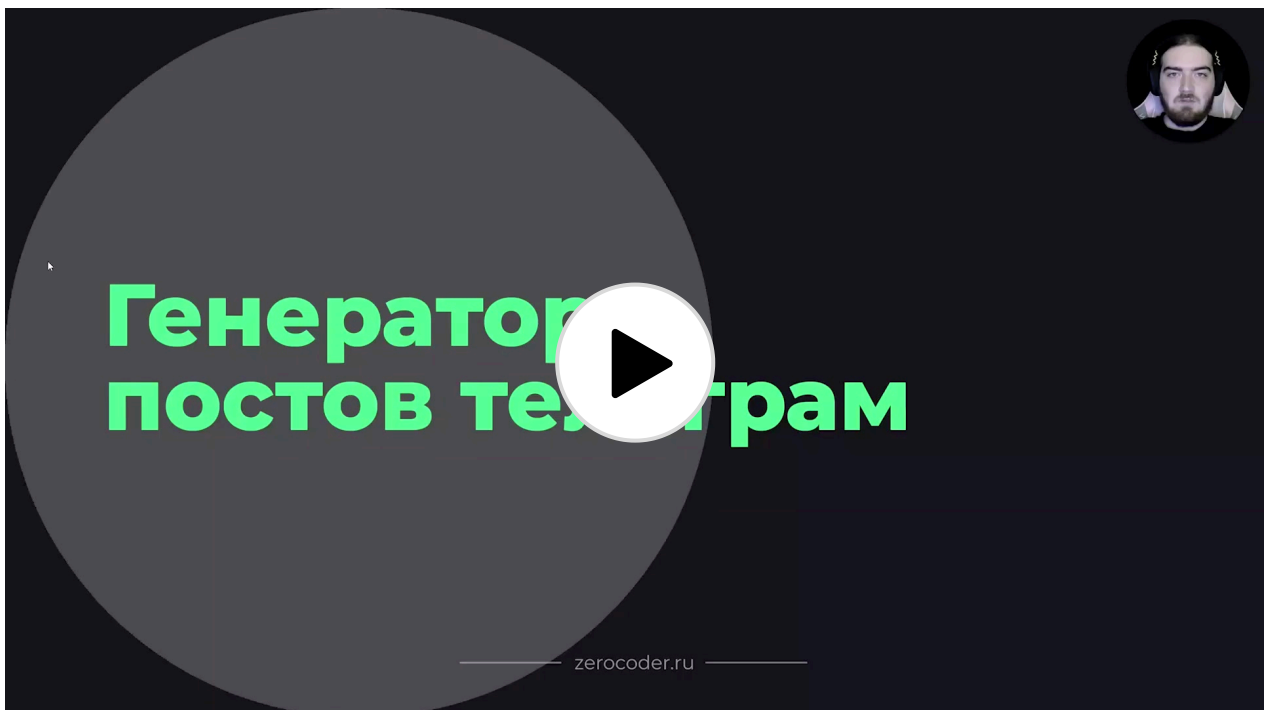
Соберешь и развернешь полноценную AI-систему автопостинга: генерация текста и изображения через OpenAI, автоматическая публикация в Telegram, защита через Authorization header и запуск на сервере 24/7.

▶ Время чтения и просмотра: ~ 1 час

🕒 Время выполнения задания: ~ 1 час

Теория на сегодня

1. Автоматизация процессов с помощью n8n



На этом уроке мы завершаем работу над генератором постов для Telegram и доводим его до финальной версии. Настроим промпты, добавим генерацию изображений, повторим ключевую теорию и закрепим работу с API, чтобы получить полноценный готовый кейс.

Действия

Сегодня мы:

1. Повторим n8n, как рабочий инструмент для автоматизации задач.
2. Повторим как мы можем делать запросы к телеграм через API.
3. Допишем небольшой n8n скрипт для автопостинга в телеграм используя openai.

▼ Автоматизация процессов в n8n

 Общая цель этого блока — научиться работать с **n8n** и создать несколько практических кейсов для портфолио.

Задача не просто разобраться в интерфейсе, а собрать реальные автоматизации, которые можно показать как готовые проекты.

 **n8n** — это платформа для создания автоматизаций. Это инструмент, который позволяет собирать автоматизированные процессы из готовых блоков.

Любая автоматизация — это работа с данными:

- данные поступают в первый блок;
- обрабатываются;
- передаются дальше;
- снова обрабатываются;
- и так до финального результата.

Почему n8n так популярен

1. Работа через UI

Всё строится через визуальный интерфейс. Процесс выглядит как схема из блоков, соединённых между собой. Не нужно писать код — логика собирается визуально.

2. Не требуется знание языков программирования

Не нужно разбираться в переменных, функциях и синтаксисе. Блоки ^{Действия} выполняют свою часть работы и передают данные дальше.

3. Визуализация процессов

При работе с кодом человек всё равно мысленно строит схему: что откуда приходит и куда передаётся. В n8n эта схема уже показана на экране. Это упрощает понимание логики и ускоряет работу.

4. Доступность и бесплатность

n8n можно запускать внутри **Docker**.



Docker — бесплатный инструмент, и n8n в **self-hosted** формате также бесплатен. Его можно развернуть локально или на удалённом сервере.

Для работы на уроке потребуется: сервер (локальный или удалённый), **Docker** и установленный n8n. Дальше вся работа строится внутри визуального интерфейса.

Главные преимущества n8n:

1. Бесплатность без жёстких ограничений

Существует множество сервисов для визуальной автоматизации, но чаще всего:

- либо отсутствует полноценный бесплатный тариф;
- либо бесплатная версия ограничена по времени;
- либо ограничена по количеству операций и ресурсов.



У n8n единственное ограничение — мощность сервера.

Если автоматизация выполняет большое количество операций, она будет упираться в вычислительные ресурсы.



n8n — приложение, работающее на компьютере или сервере с конечными ресурсами.

2. Эффект популярности и сообщества

n8n популярен во многом благодаря сообществу. Существует большое количество готовых сценариев (**workflow**), которые пользователи публикуют бесплатно.

Это создаёт эффект «снежного кома»:

- пользователи создают решения;
- делятся ими;
- новые пользователи приходят благодаря этим решениям;
- появляется ещё больше готовых сценариев.

Для практической работы это большой плюс.

Если возникает задача, которую сложно собрать с нуля, можно:

- найти готовую заготовку;
- изучить структуру блоков;
- адаптировать под свои задачи;
- объединить несколько решений в один проект.

Таким образом обучение происходит на реальных рабочих примерах.

3. Простота входа

Для работы с **n8n** не требуется серьёзное знание программирования. Не нужно писать код, отлаживать его вручную и запускать отдельно. Всё строится из блоков, которые последовательно обрабатывают данные.

Это делает инструмент удобным для:

- автоматизации простых процессов;
- создания **MVP-решений**;
- проектов, где не хочется уходить в полноценную разработку.

В сегодняшнем кейсе соберем простую схему, где принимается сообщение → выполняется несколько запросов → результат обрабатывается → данные отправляются в **Telegram** → **Telegram** публикует пост в канале. Это базовый, но полноценный пример автоматизации, который можно добавить в портфолио как готовый кейс.

1. Визуальный конструктор автоматизаций и интеграций

✓ **n8n** — это инструмент для сборки автоматизаций через визуальный интерфейс. Его часто относят к **low-code** решениям.

При этом корректнее называть его именно **low-code**, а не **no-code**, потому что:

- существуют блоки с выполнением кода;
- используются переменные;
- необходимо базовое понимание логики работы данных.

Минимальное понимание того, что такое переменные и как передаются данные между шагами, всё же требуется.

2. Готовое решение для бизнеса

Внутри **n8n** доступно большое количество готовых **workflow** — сценариев автоматизации, которые можно:

- просматривать;
- копировать;
- адаптировать под свои задачи;
- объединять между собой.

Это сильно ускоряет разработку и позволяет учиться на готовых рабочих примерах.

3. Ключевая концепция — Workflow

✓ **Workflow** — это последовательность шагов, которые выполняются для достижения результата.

Проще говоря:

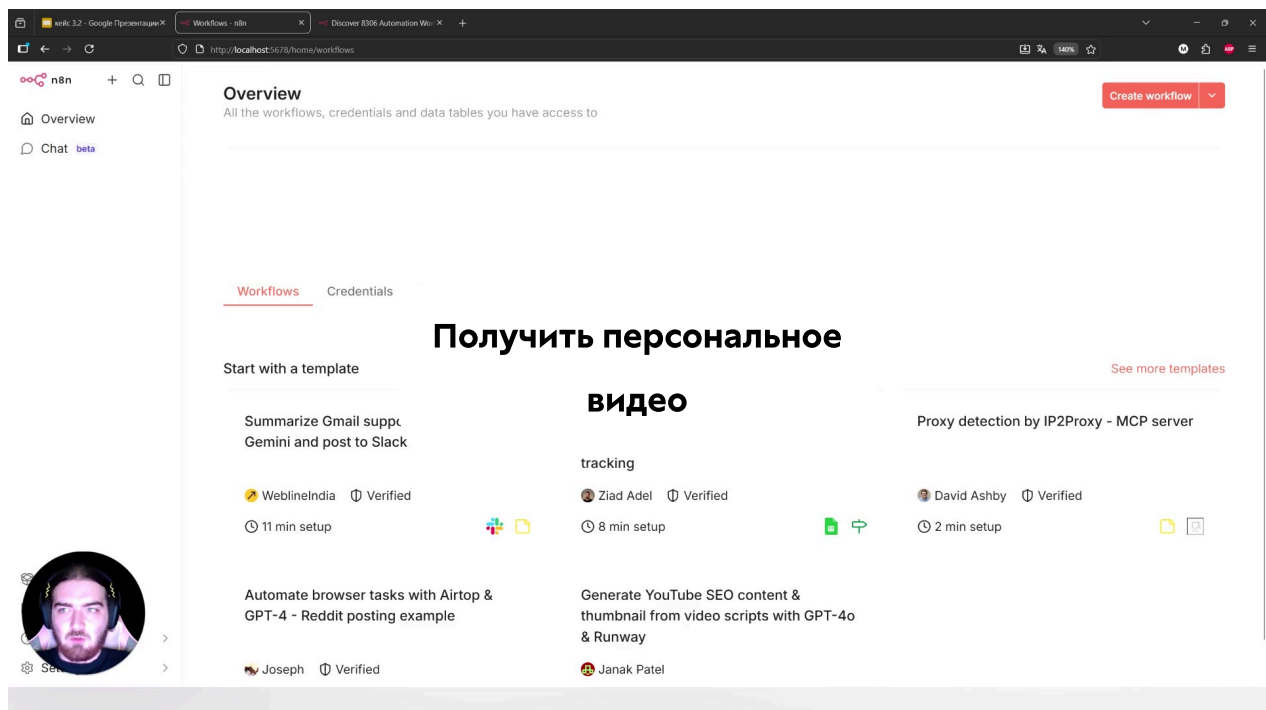
- что происходит сначала;
- что происходит после;
- какие данные передаются;
- какой финальный результат должен быть получен.

⚠️ Вся логика проекта в **n8n** — это правильно выстроенный **workflow**.

Переходим к практике:

- разберем уже собранную схему;
- рассмотрим примеры готовых проектов внутри **n8n**;
- доработаем текущий **workflow**;
- перенесём проект на сервер;
- разберем, зачем вообще переносить автоматизацию на удалённый сервер и в чём практический смысл этого шага.

2. Запуск n8n, создание и доработка схемы



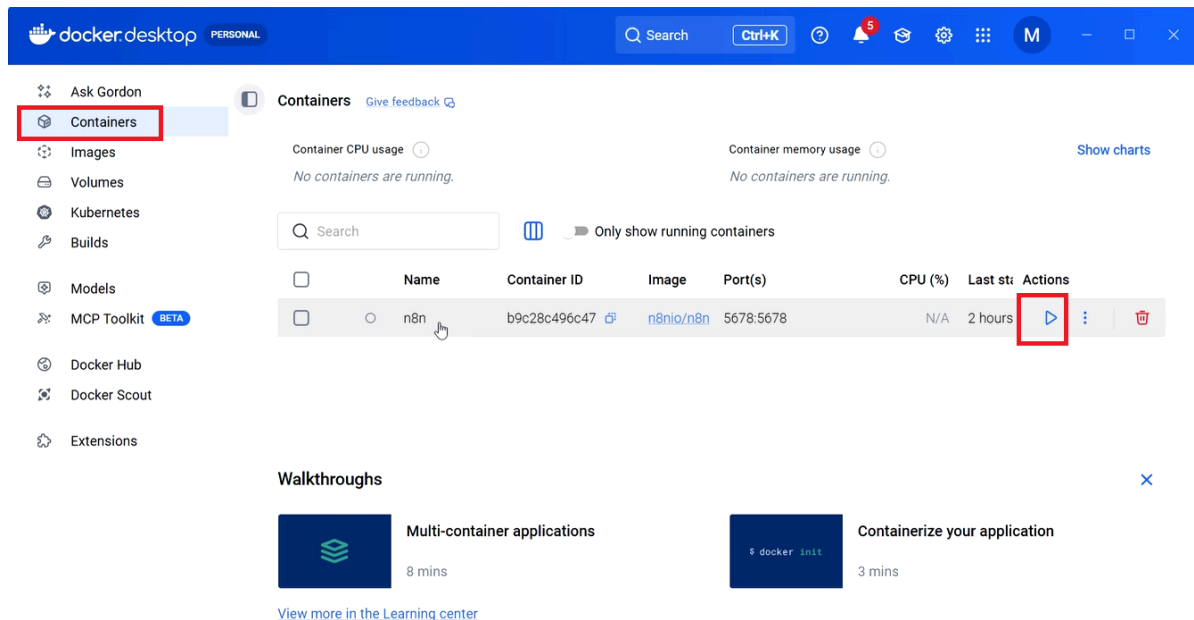
▼ Запуск и интерфейс n8n

Если **n8n** недоступен по адресу: страница загружается, но интерфейс не открывается — это означает, что контейнер не запущен.

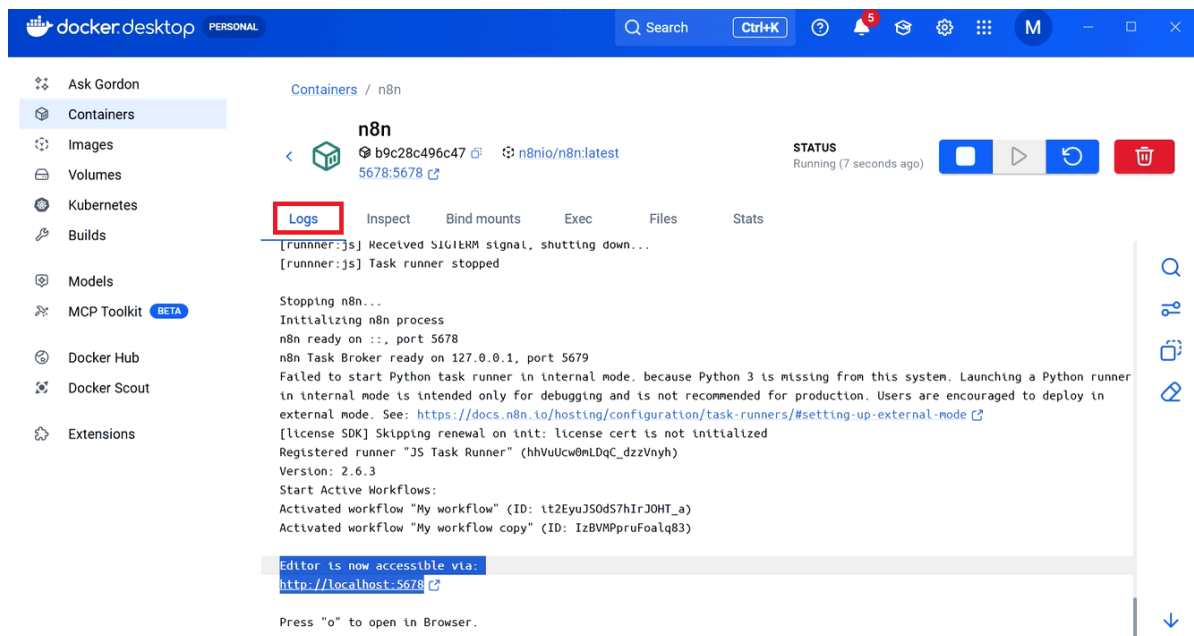
1. Запуск n8n через Docker

Действия

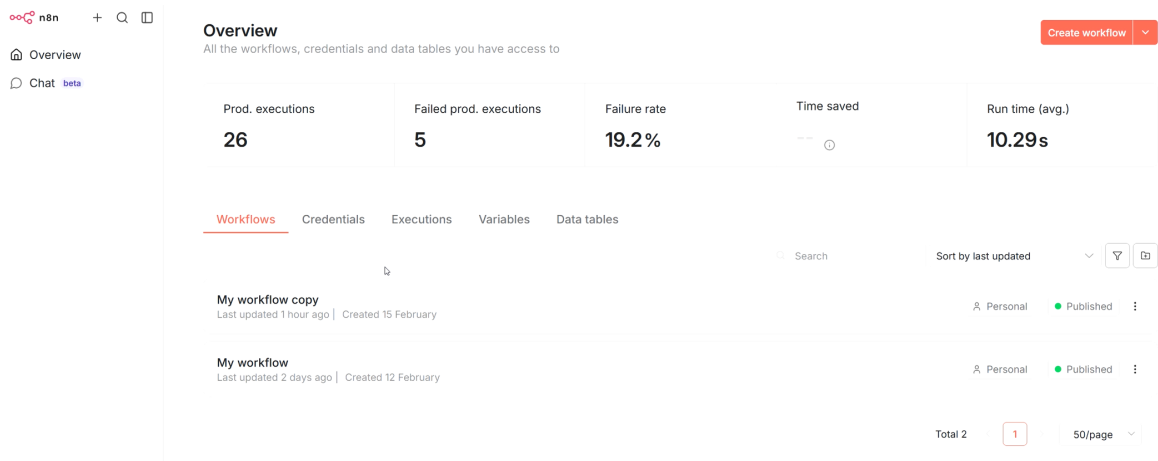
- Переходим в **Docker** → в левом меню выбираем раздел Containers → в списке контейнеров отображается один или несколько контейнеров → выбираем тот контейнер, в котором ранее настраивалась работа.
- Справа от названия контейнера нажимаем кнопку Start → контейнер запускается. После запуска требуется немного времени для инициализации.



- Нажимаем левой кнопкой мыши на контейнер **n8n** → переходим в раздел Logs. В логах отображается процесс запуска. Появляется строка с информацией, что **n8n** стал доступен с компьютера.

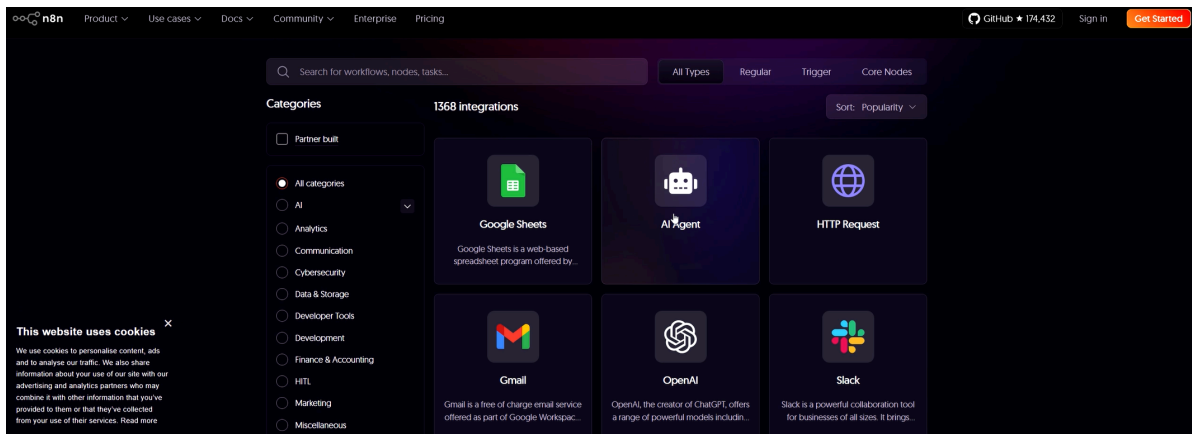


- Если после запуска интерфейс не отображается, необходимо подождать 5–15 секунд — сервису требуется время для запуска. Переходим обратно в браузер → обновляем страницу. После обновления отображается интерфейс **n8n**. Страница полностью загружается и система готова к работе.
Действия



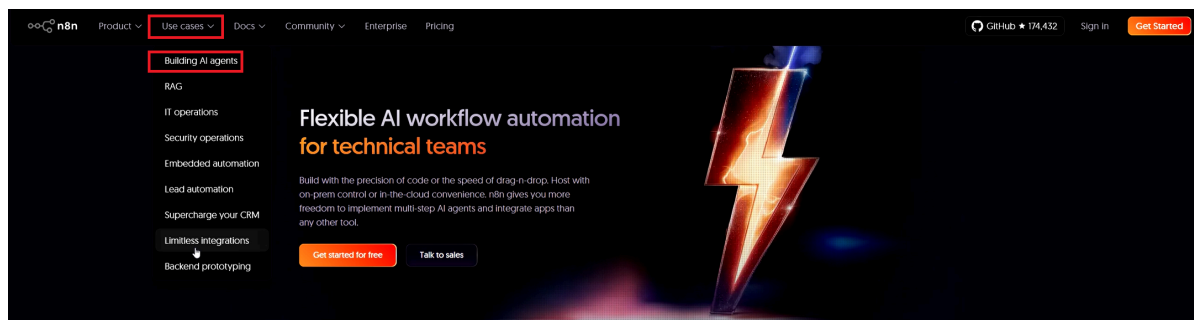
2. Интерфейс n8n:

- Переходим в интерфейс **n8n** → открываем главную страницу. С главной страницы доступен переход в раздел интеграций.
- Переходим в раздел **Integrations** → открывается список доступных интеграций. В списке отображаются различные сервисы и инструменты: **HTTP Request, Google Sheets, AI-агенты** и множество других систем. Список интеграций очень большой. Всего доступно около 1400 интеграций, с которыми можно работать. В верхней части списка представлены основные и наиболее популярные интеграции.

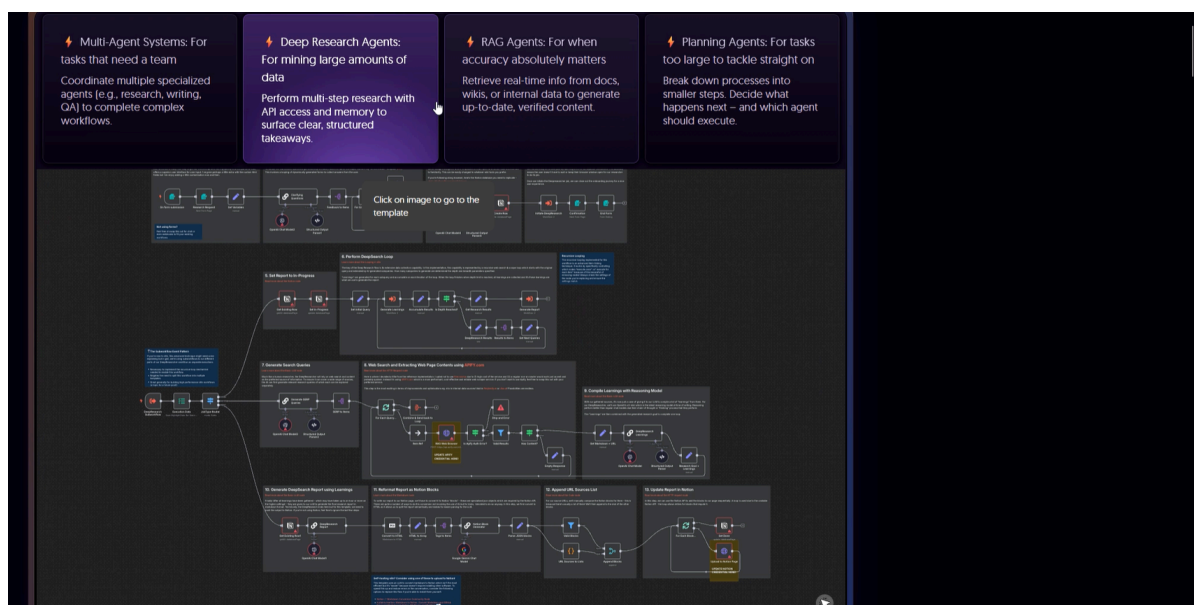


✓ **HTTP Request** является универсальным инструментом, с помощью которого можно реализовать практически любые сценарии взаимодействия с внешними сервисами.

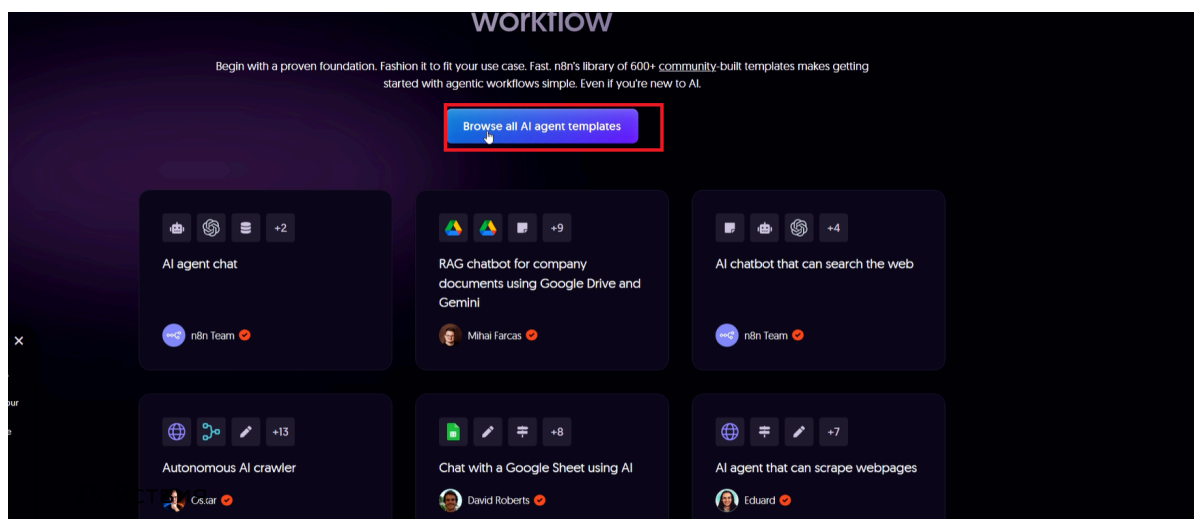
- С главной страницы нажимаем **Use cases** → выбираем интересующий сценарий, например, выбираем **Build AI agent**.



- После выбора раздела листаем страницу вниз. Отображаются примеры, предоставленные самим **n8n**. Интерфейс и описания представлены на английском языке. При необходимости можно воспользоваться переводом. В примерах наглядно показана структура сценариев: какие блоки используются, откуда идут данные, куда они передаются, с какой целью. Можно визуальнo увидеть логику построения процесса.

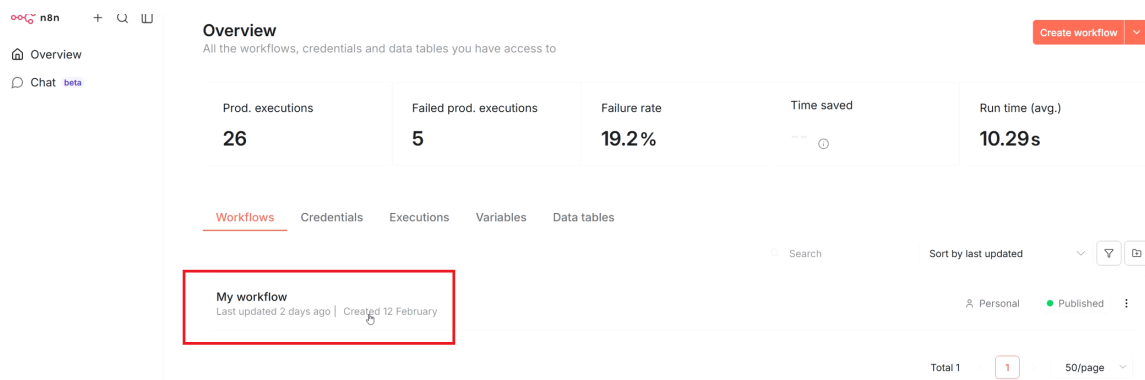


- Прокручиваем страницу ниже → находим кнопку **Browse all AI agent templates** → нажимаем → листаем ниже → отображается большое количество готовых шаблонов. Это заготовки, которые можно: использовать напрямую, адаптировать под свои задачи, изучить структуру и логику, взять отдельные идеи и решения для собственных проектов.



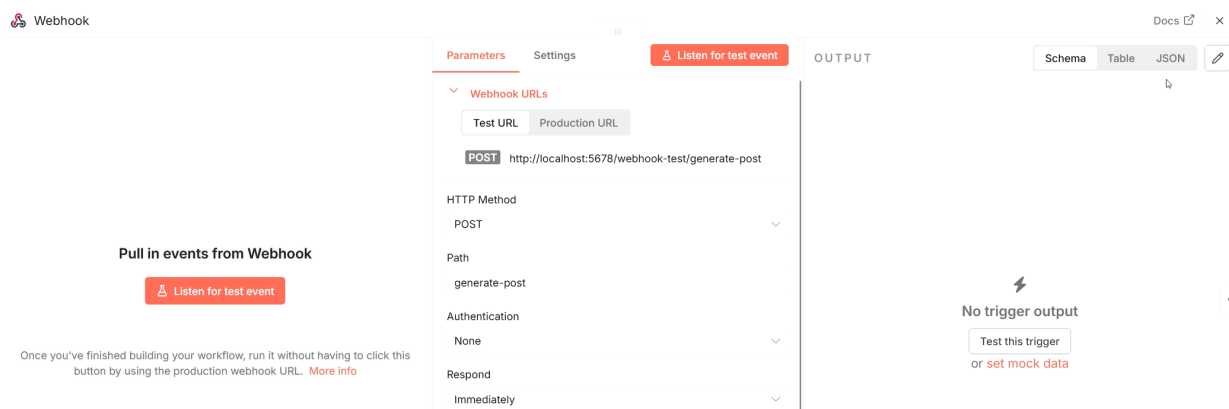
3. Просмотр созданного workflow:

- Возвращаемся в интерфейс **n8n** → проверяем, что сайт работает, все активно → открываем **My workflow**, который ранее был создан.

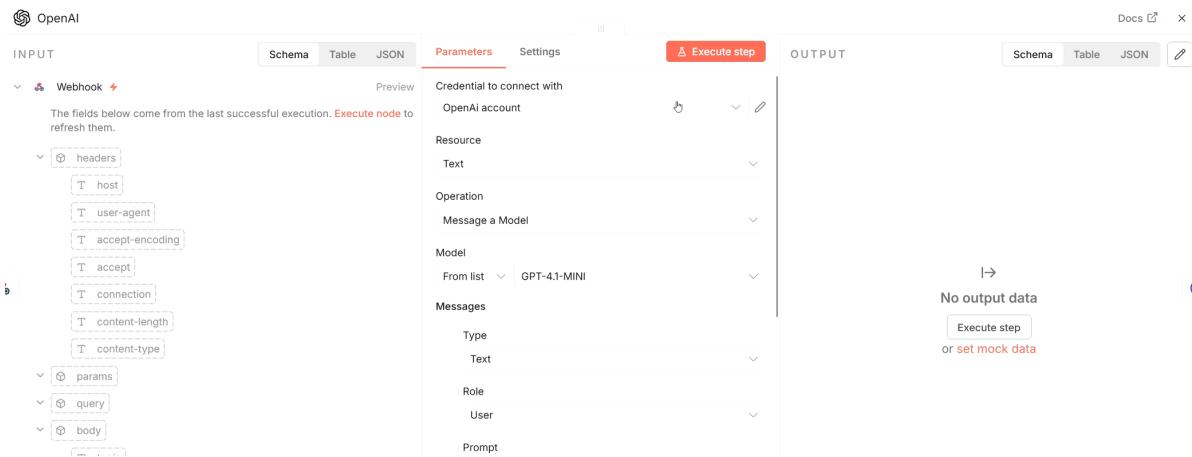


- В структуре используется три максимально простых блока: **Webhook**, **OpenAI**, **Telegram**.
- Webhook** принимает запросы извне → используются стандартные настройки → **URL** формируется системой автоматически → метод запроса — **POST** → путь — **generate post**.

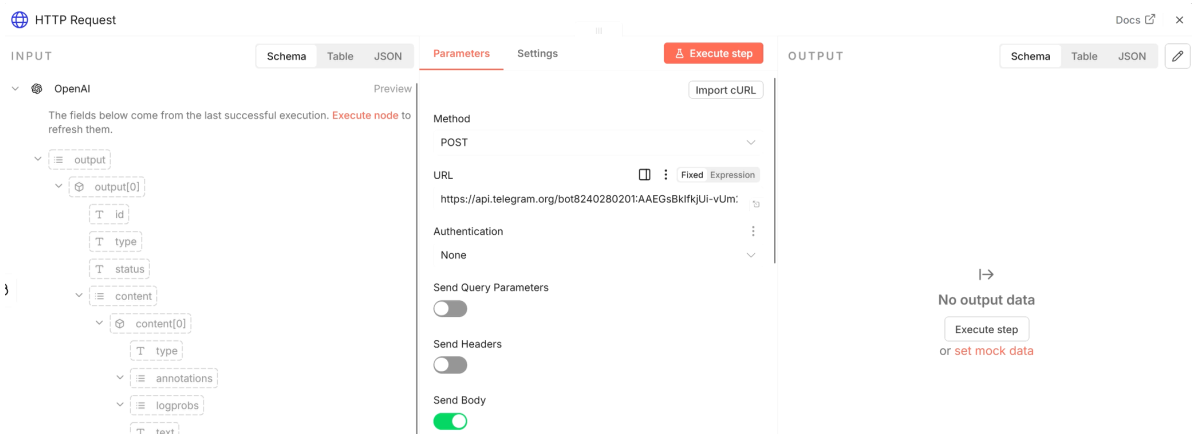
⚠ Путь можно изменить на любой собственный. Это окончание **URL**, которое добавляется к основному адресу. Если изменить название пути, система сразу отображает изменения в **URL**.



- В блоке **OpenAI** подключен доступ к аккаунту **OpenAI**. Для подключения создаются **credentials** → вводится **API-ключ** → выполняется тест соединения. После успешной проверки соединения доступ можно использовать. В блоке используется простой промпт для генерации текста поста в **Telegram**.

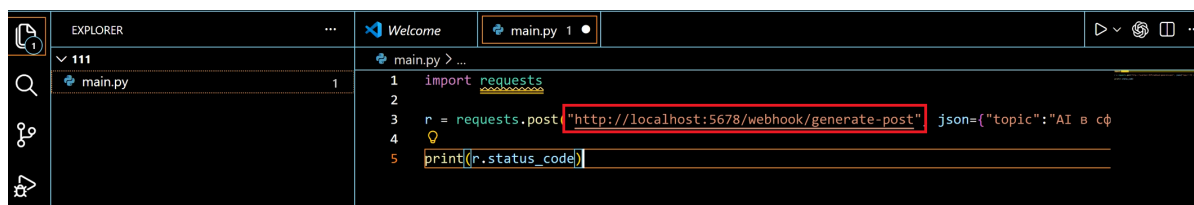


- В блоке **Telegram** используется действие **Send Message**. Происходит отправка сгенерированного текста в **Telegram**.



4. Тестирование через Visual Studio:

- Переходим в **Visual Studio** → в коде заменяем путь на свой. Запускаем код.



Если используется **Visual Studio**, **Cursor** или аналогичный редактор, необходимо вручную сохранять изменения в файле. Если рядом с названием файла отображается белая точка вместо крестика, это означает, что в файл внесены изменения, но они не сохранены.

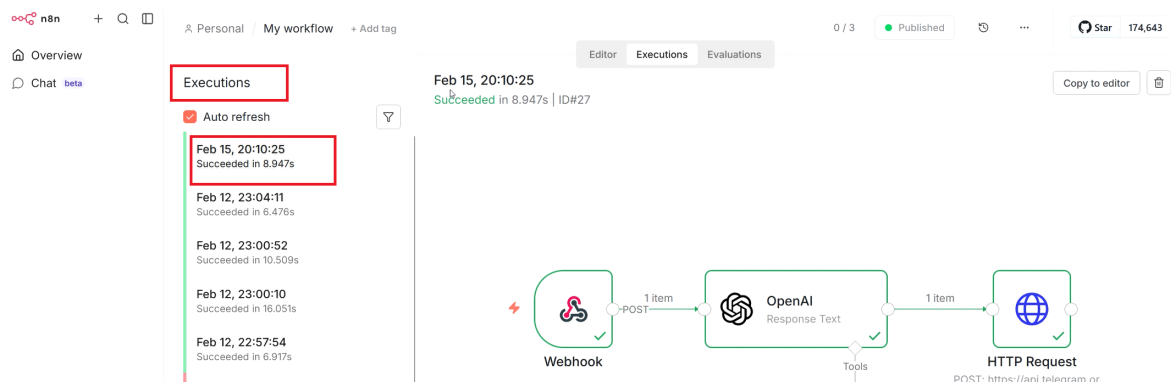


Действия

- Чтобы сохранить файл нажимаем `Ctrl + S`. После сохранения снова запускаем код. После запуска отображается статус `200`.

✓ Статус-код 200 означает, что запрос успешно дошел до сервера и получен ответ.

- Переходим в раздел **Executions**, где отображается текущее выполнение. В списке видно выполнение с текущей датой и временем. Это подтверждает, что запрос был успешно обработан.

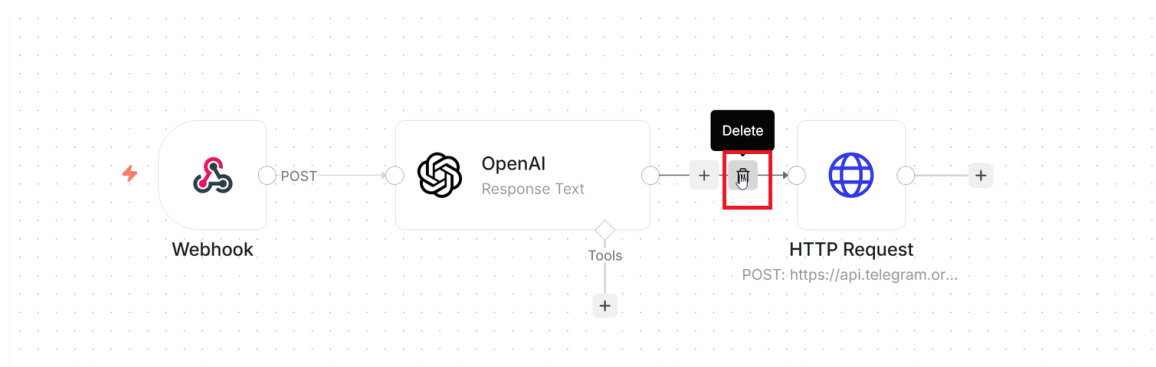


- Открываем **Telegram**. Проверяем, отображается ли текст, сгенерированный искусственным интеллектом и отправленный системой.

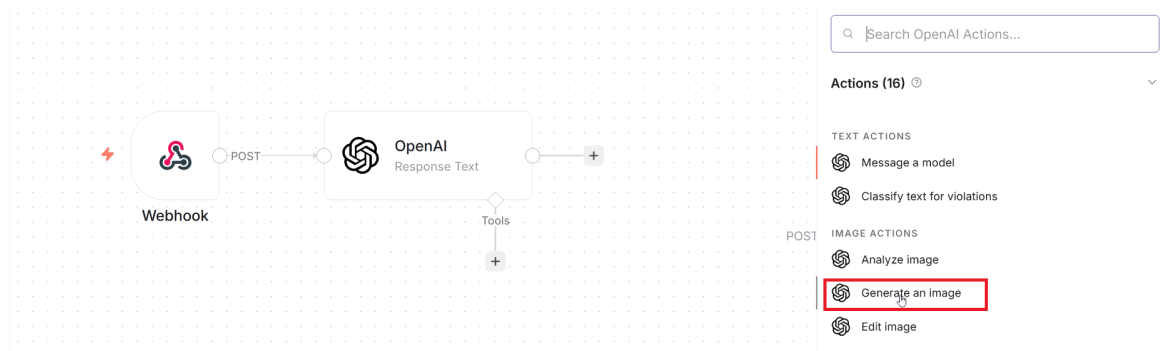
▼ Доработка схемы

1. Обновление структуры workflow

- Удаляем текущее соединение между блоками. Наводим курсор на соединительную стрелку → нажимаем **Delete** → связь удаляется.



- Нажимаем плюс справа от предыдущего блока → выбираем направление вправо → из списка инструментов выбираем раздел **AI** → ниже выбираем **OpenAI** → выбираем действие **Generate an image**.

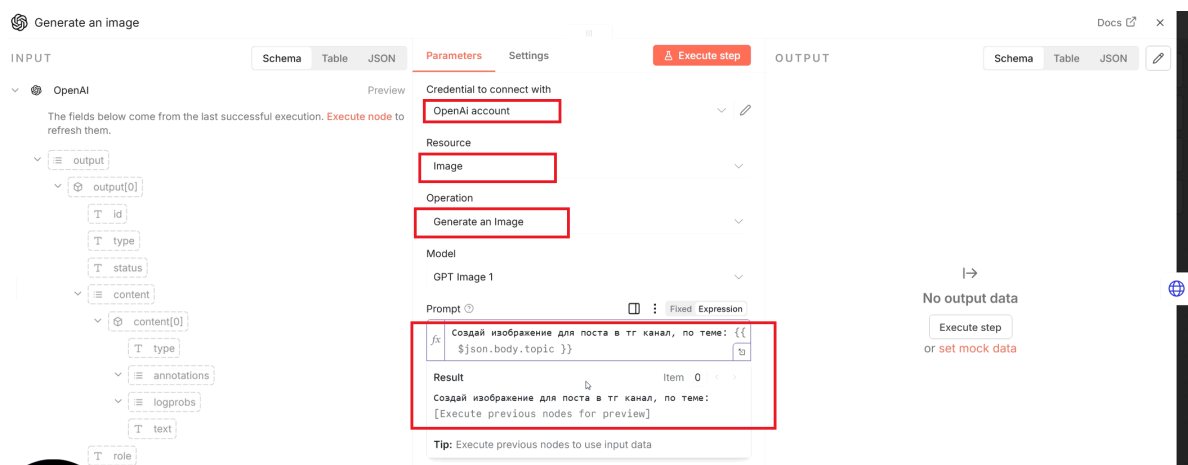


- Аккаунт OpenAI подставляется автоматически (используются сохраненные **credentials**). В настройках в поле Resource указываем Image → в поле Operation выбираем Generate an Image → выбираем модель GPT Image1 → в поле Prompt указываем текст запроса.

 *Создай изображение для поста в тг канал, по теме: {{ \$json.body.topic }}*

Result

*Создай изображение для поста в тг канал, по теме:
[Execute previous node for preview]*

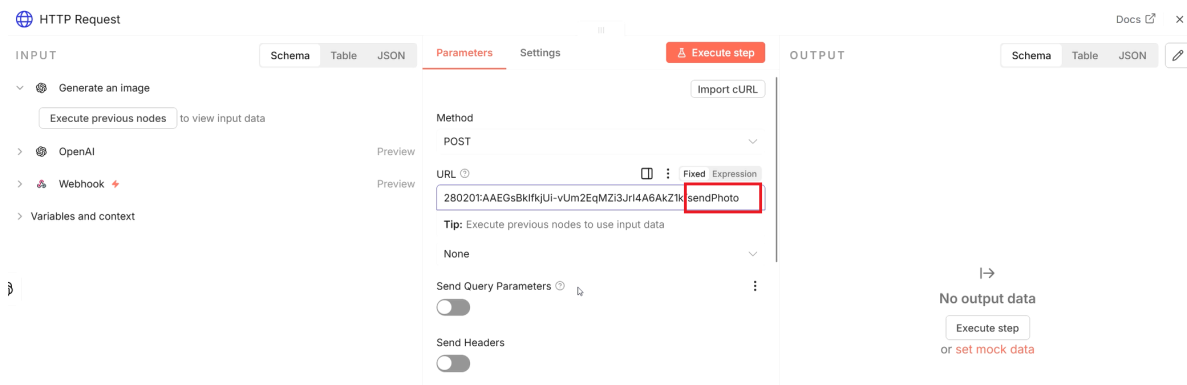


- Блок генерации изображения соединяем с **HTTP-запросом**.

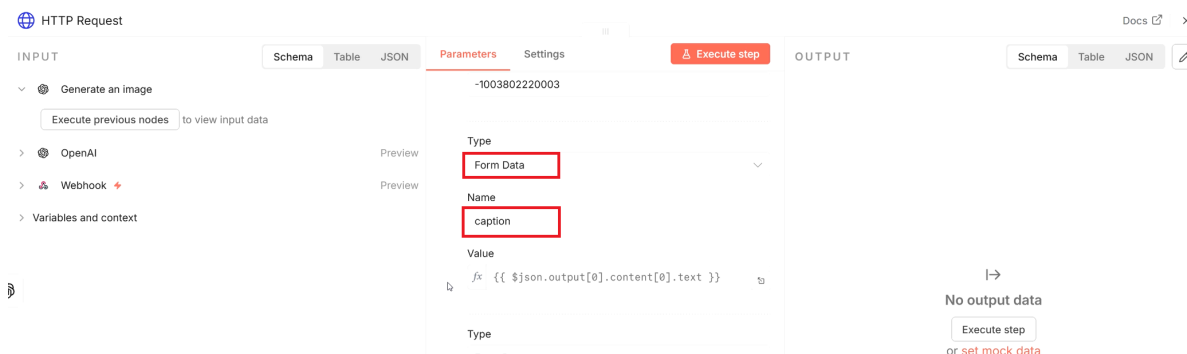
2. Настройка HTTP Request:

- Необходимо изменить **URL**. Ранее использовался sendMessage для отправки текста. Для отправки изображения меняем конец URL с sendMessage → на sendPhoto .

Действия



- В поле Name, которое ранее называлось text меняем на caption.



При отправке текста используется поле text. При отправке фото используется поле caption — это описание или подпись к изображению. Чтобы текст корректно отображался в **Telegram**, обязательно указываем caption.

- Изменяем источник данных, чтобы корректно получать результат. Для этого указываем квадратные скобки и внутри прописываем название ноды **OpenAI** в двойных кавычках, после названия ноды указываем путь к нужному полю `{ $node["OpenAI"].json.output[0].content[0].text }`



Нода — это отдельный исполнительный блок.



- Добавляем новый Body Field для HTTP-запроса.

Действия

Порядок Body Field не имеет значения, в **JSON** все будет упаковано как словарь ключ-значение. Пример структуры JSON :
output → id, type, status, content. Можно обращаться напрямую к нужному ключу: например, к content → текст.

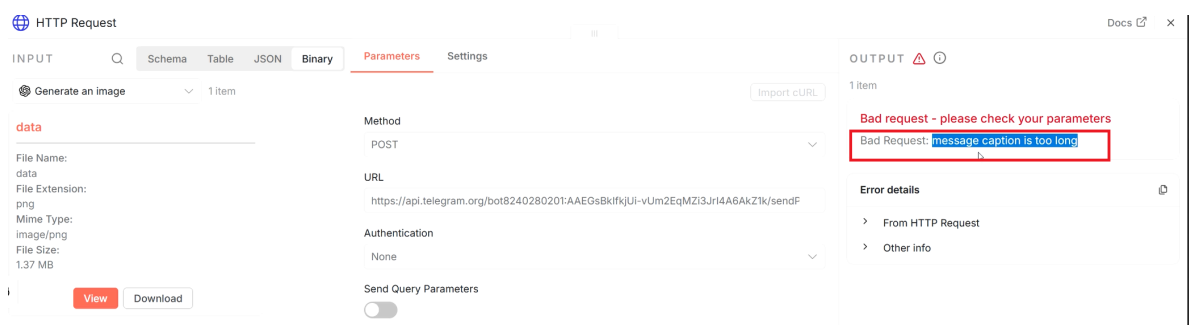
- Указываем тип данных: n8n Body Field → название: photo (это то изображение, которое будет отправлено) → input Data Field Name : data → источник бинарных данных (разберется при запуске). После этих настроек запрос будет отправляться по пути sendPhoto, и изображение будет корректно передаваться.



- Настройка завершена → закрываем блок.

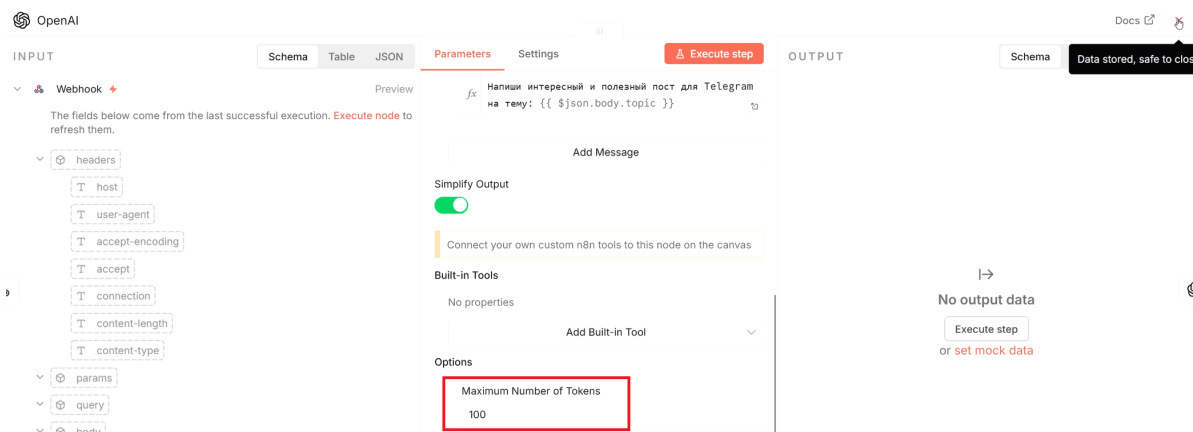
3. Тестирование обновленного workflow

- Запускаем код → нажимаем Publish (опубликовать изменения) → выполняем команду python main.py.
- Переходим в раздел Executions → проверяем новое выполнение. Процесс занимает больше времени, так как теперь генерируется не только текст, но и изображение. Генерация текста происходит быстро, генерация изображения требует больше времени. Ожидание может занять 20–30 секунд, иногда до минуты. Время зависит от скорости работы модели.
- Если отображается ошибка в последнем блоке, например Message caption is too long, это означает ограничение **Telegram**. При отправке фото текст не должен превышать 1024 символа.



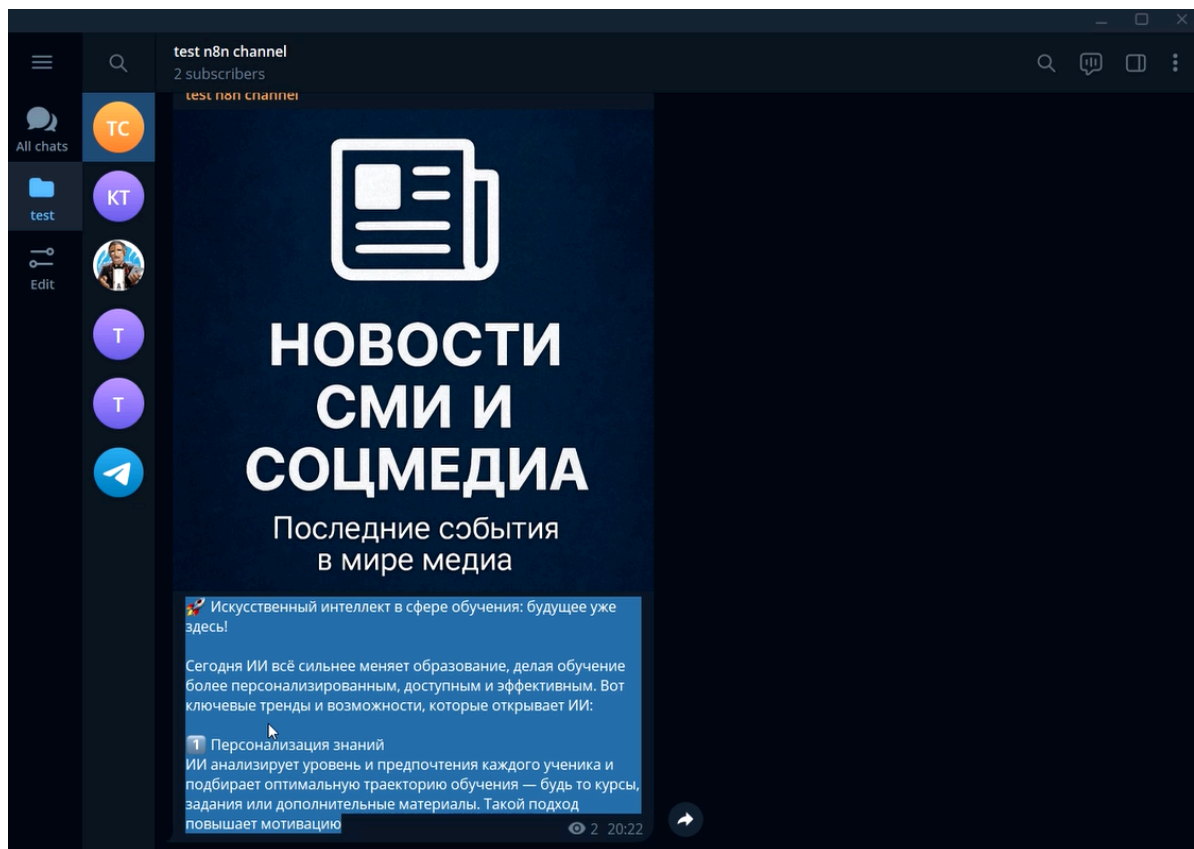
- Открываем ноду **OpenAI** → листаем вниз → находим раздел Options → нажимаем Add Option → выбираем Maximum Number of Tokens →

устанавливаем ограничение на количество токенов в ответе модели, например 100 токенов для коротких постов.



Один токен — это не одно слово. Обычно один токен равен 2–5 символам. Если установить 250 токенов, текст, скорее всего, поместится в 1024 символа. Однако в редких случаях может превысить лимит. Для гарантированного результата можно установить 200 токенов или 100 токенов. Значения выше 250 токенов почти гарантированно превысят лимит 1024 символа.

- Сохраняем → закрываем → нажимаем Publish.
- Снова запускаем python main.py → переходим в Executions → ожидаем завершения → переходим в **Telegram**. Текст получился коротким, так как установлено ограничение 100 токенов. Из-за ограничения текст может обрываться на полуслове или половине фразы из-за того, что токены закончились. Для корректной завершенности текста потребуется доработка промпта.



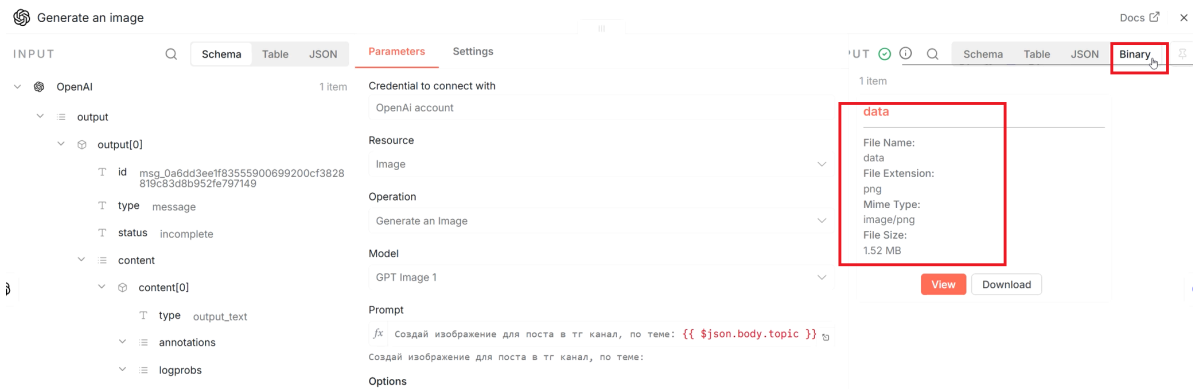
Несмотря на ошибки, система работает корректно: все блоки выполняются последовательно, изображение генерируется, текст отправляется, логика **workflow** отрабатывает полностью.

⚠ Ошибки — нормальная часть работы. Их не нужно бояться или останавливать проект. Необходимо искать решение и корректировать настройки.

Тонкая настройка промпта и улучшение качества текста — следующий этап работы.

5. Передача **binary**-файла в HTTP-запрос:

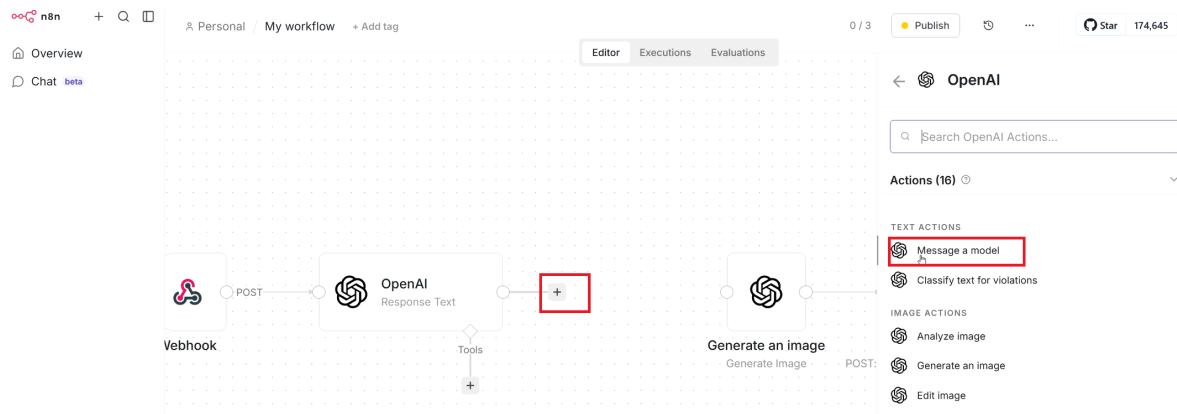
- Открываем ноду **Generate an Image** → переходим в раздел **Output**, где отображается **Binary** — это тот самый бинарный файл, который передается далее в **Telegram**.
- Внутри **Binary** присутствует поле **data**. Поле **data** содержит всю необходимую информацию: имя файла, расширение файла, название файла, размер файла.



- В этом объекте содержатся все данные, которые необходимо передать **Telegram**, чтобы он принял изображение и корректно его обработал.

6. Добавление блока генерации промпта для изображения

- Нажимаем плюс → выбираем AI → OpenAI → Message Model → выбираем модель GPT 4.1.mini.



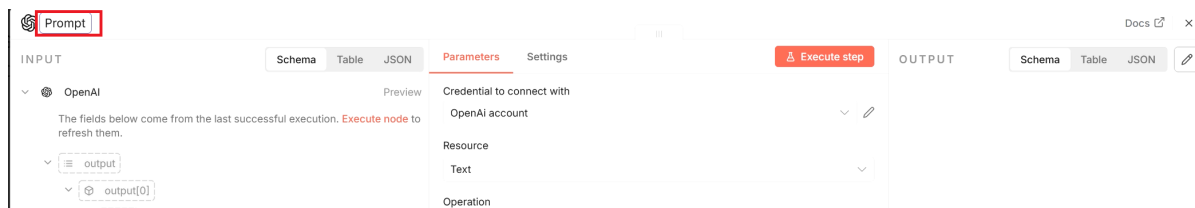
- Устанавливаем роль System → в системном сообщении прописываем инструкцию, которая определяет поведение модели:

 *Используя переданное содержимое поста для телеграм, напиши промт для GPT_IMAGE 1, для генерации обложки поста.*

- Нажимаем Add Message → выбираем роль User → в тексте указываем содержимое поста, полученное из предыдущей ноды. Используем обращение через ноду: `{ $node["Prompt"].json.output[0].content[0].text }`. Предварительно проверяем путь в HTTP Request.
- Ограничение по токенам не устанавливаем, так как результатом будет текст промпта для генерации изображения, а итогом всей цепочки все равно будет изображение. Закрываем блок.
- Элементы схемы визуально выравниваем: соединяем блок генерации промпта с блоком генерации изображения.

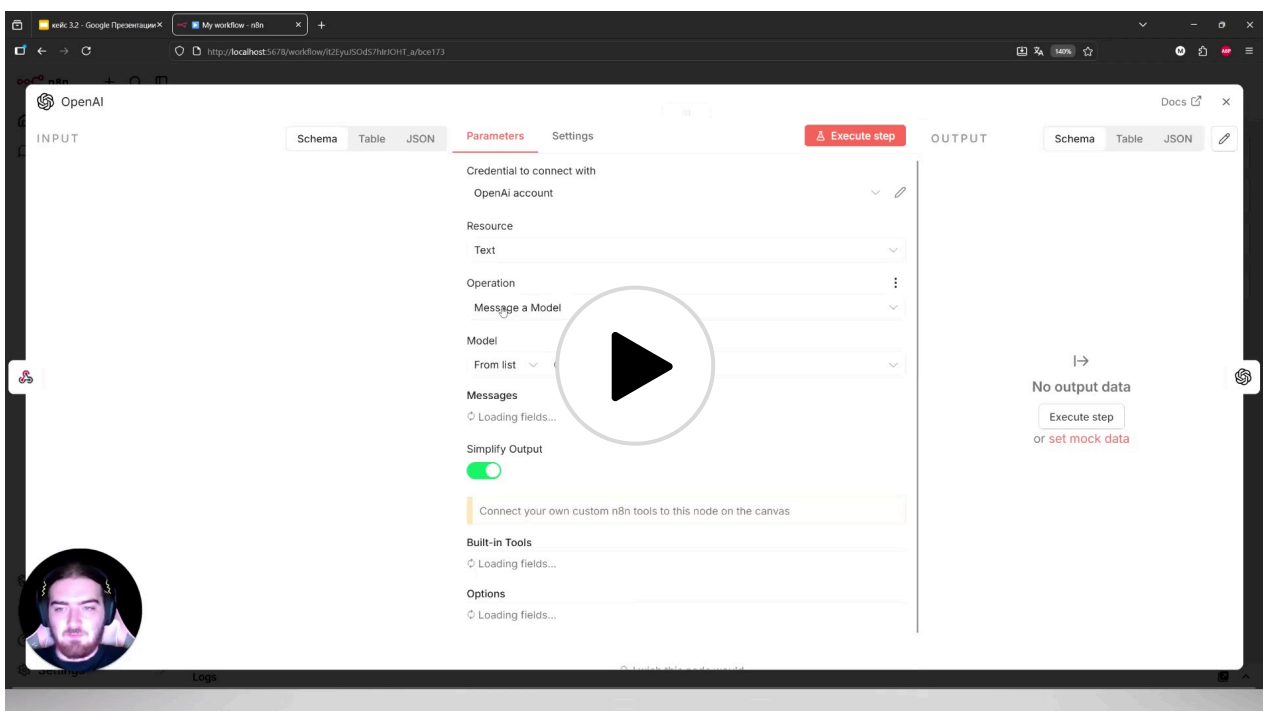
Действия

- Переименовываем блок Message Model в поле: названия сверху указываем Prompt (генератор промпта).



- Открываем блок Generate an Image → в поле Prompt вместо статичного текста указывается обращение к новой ноде:
`{ $node["Prompt"].json.output[0].content[0].text }`. Дополнительный текст вручную не добавляем, так как на выходе уже формируется готовый промпт. Закрываем блок.
- Нажимаем Publish → подтверждаем публикацию → запускаем **workflow** → открываем Executions → проверяем выполнение процесса.
- Текущее ограничение по токенам слишком маленькое, из-за этого пост получается коротким и недостаточно содержательным. Для более качественного результата необходимо увеличить лимит. Можно добавить дополнительно 50–100 токенов. Слишком маленький объем текста не позволяет посту быть содержательным и полноценно раскрывать тему.

3. Настройка промптов и защиты системы



Переходим к финальной донастройке промптов.

1. Увеличение лимита токенов в генерации текста

- Открываем первый блок **OpenAI**, где генерируется текст → увеличиваем Maximum Tokens до 250.
- Средний размер одного токена примерно 3–4 символа. Лимит 250 токенов должен укладываться в ограничения **Telegram** и при этом давать более содержательный текст.

2. Перестройка структуры сообщений

- Дорабатываем структуру: оставляем User → добавляем еще одно сообщение → в первом сообщении меняем роль с User на System → старый промпт удаляем → вставляем новый подготовленный промпт.

 Ты профессиональный редактор Telegram-каналов и копирайтер.

Твоя задача — писать короткие, естественные и завершённые Telegram-посты.

Строгие ограничения:

- Максимальный размер ответа: 250 токенов
- Текст должен быть ЗАВЕРШЁННЫМ, нельзя обрывать мысли
- Если не помещается в лимит — сокращай формулировки
- Никаких незаконченных предложений
- Пост должен выглядеть цельным и логически завершённым

Стиль и формат:

- Живой, человеческий язык
- Без воды, длинных вступлений и канцелярита
- Без фраз вроде "Сегодня поговорим", "В современном мире"
- Допустимы умеренные эмодзи
- Короткие абзацы приветствуются
- Структура: хук → ценность → завершение

Пиши так, чтобы пост выглядел как реальная публикация в Telegram.

- Во втором сообщении оставляем роль User — там передается тема. Действия

3. Стиль/материалы/свет

4. Ограничения: "no text, no watermark, no logo"

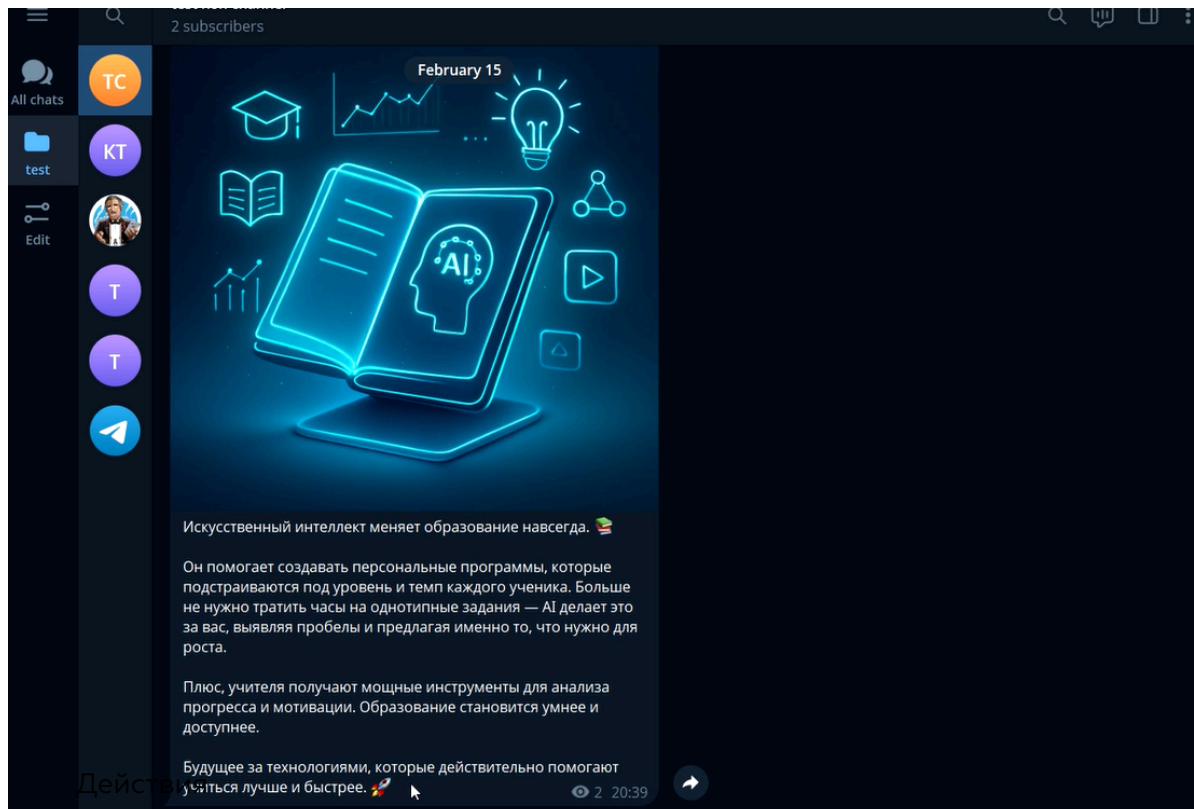
- User оставляем для передачи текста поста.



- Закрываем блок → нажимаем Publish.

4. Тестирование

- Запускаем выполнение.
- Переходим в Executions → проверяем запуск процесса. Ожидаем завершения.
- Переходим в **Telegram** → проверяем результат. Получается более длинный текст, чем ранее, но при этом мысль завершена, текст не обрывается, структура логичная. Пост выглядит корректно и цельно.



- Система полностью работает: генерируется текст, создается промпт для изображения генерируется изображение, отправляется пост в Telegram. Все блоки выполняются корректно и последовательно.

⚠️ Использованные промпты можно изменять и адаптировать под свои задачи. Нет строгого требования использовать их дословно. Можно: менять стиль, добавлять ограничения, усиливать структуру, адаптировать под собственный формат.

▼ Настройка защиты 📌

Перед развертыванием на сервере необходимо настроить защиту, так как сейчас система не использует методы авторизации. Если загрузить проект на сервер в текущем виде, любой, кто узнает **IP-адрес** и путь запроса, сможет отправлять запросы.

1. Открываем первую ноду **Webhook** → находим поле Authentication → выбираем Header Auth → появляется настройка заголовков → нажимаем Create new credentials.

The screenshot shows the configuration for a Webhook node. The 'Parameters' tab is active. Under 'Webhook URLs', the 'Test URL' is selected, showing a POST request to 'http://localhost:5678/webhook-test/generate-post'. The 'HTTP Method' is 'POST' and the 'Path' is 'generate-post'. The 'Authentication' section is expanded, showing 'Header Auth' selected. Below this, the 'Credential for Header Auth' dropdown shows 'Header Auth account' selected. A red box highlights the '+ Create new credential' button. To the left, a sidebar shows 'Pull in events from Webhook' with a 'Listen for test event' button. To the right, the 'OUTPUT' tab shows 'No trigger output' with a 'Test this trigger' button.

2. В поле Name указываем: Authorization → в поле Value вставляем значение ключа.

Connection
Sharing
Details

Need help filling out these fields? [Open docs](#)

Name

Authorization

Value

.....

To send multiple headers, use a "Custom Auth" credential instead

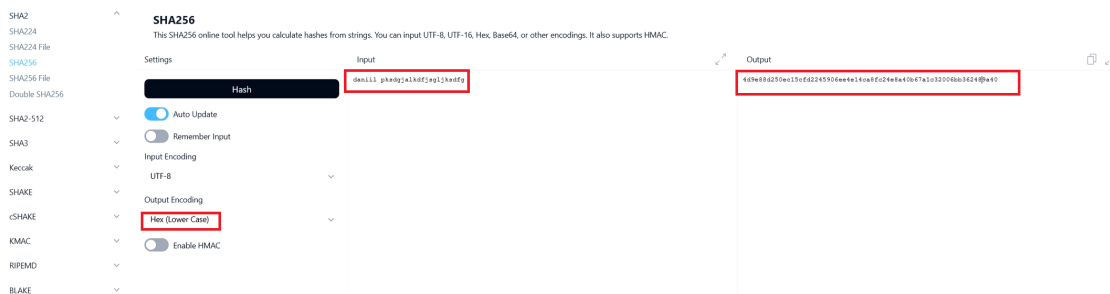
Allowed HTTP Request Domains

All

Enterprise plan users can pull in credentials from external vaults. [More info](#)



Совет: Написать в блокноте слово **Bearer** → сгенерировать длинную случайную строку через любой **Hash Generator** → использовать строку из букв и цифр (лучше **Lower Case** или смешанный формат).



3. Копируем сгенерированную строку → в блокноте формируем: Bearer пробел сгенерированный ключ → копируем полностью эту строку и вставляем в поле Value → сохраняем.



Важно сохранить этот ключ. Если он будет утерян, доступ к **Webhook** получить не получится.

4. Если выполнить запрос без заголовка авторизации, возвращается ошибка 403 Unauthorized. Это означает отсутствие авторизации.

The screenshot shows a VS Code editor with a file named `main.py`. The code in the editor is as follows:

```
1 import requests
2
3 headers = {
4     "Content-Type": "application/ison",
5     "Authorization": "Bearer 4d9e88d250ec15cfd2245906ee4e14ca8fc24e8a40b67a1c32006bb362"
6 }
7
8 r = requests.post("http://localhost:5678/webhook/generate-post", json={"topic": "AI в сф
9
10 print(r.status_code)
```

The `"Authorization"` header value is highlighted with a red box. Below the editor, the `TERMINAL` tab is active, showing the command `python .\main.py` being executed three times. The first two executions returned `200`, and the third execution returned `403`, which is also highlighted with a red box.

5. В коде добавляем `headers` указываем: `Content-Type: application/json`, `Authorization: Bearer` сгенерированный_ключ. Заголовок `Authorization` должен полностью совпадать с тем, что был указан в **Webhook**. Итоговая структура запроса:

```
import requests #library for API requests
#Request headers/security
headers = {
    "Content-Type": "application/json",
    "Authorization": "Bearer 4d9e88d250ec15cfd2245906ee4e14ca8fc24e8a40b67
}
#Request to n8n for generation
r = requests.post("http://213.139.229.183:5678/webhook/generate-post", jso
#Output status code from n8n:
#200 - success, everything went as expected
#404 - wrong url
#403 - problem with headers, no access
#Any other error code - https://developer.mozilla.org/en-US/docs/Web/HTTP/
#We search for our error number and see what went wrong
print(r.status_code)
```

6. В настройках **Webhook** можно изменить параметр `Allow HTTP Request Domains`. Можно оставить `All` → указать конкретный **IP** или домен (если запросы выполняются только с одного источника) По Действия

умолчанию оставляем All , так как длинный сгенерированный ключ обеспечивает высокий уровень защиты.

Header Auth account 2
Header Auth

Save ×

Connection

Sharing

Details

Need help filling out these fields? [Open docs](#)

Name

Authorization

Value

.....

To send multiple headers, use a "Custom Auth" credential instead

Allowed HTTP Request Domains ⓘ

All

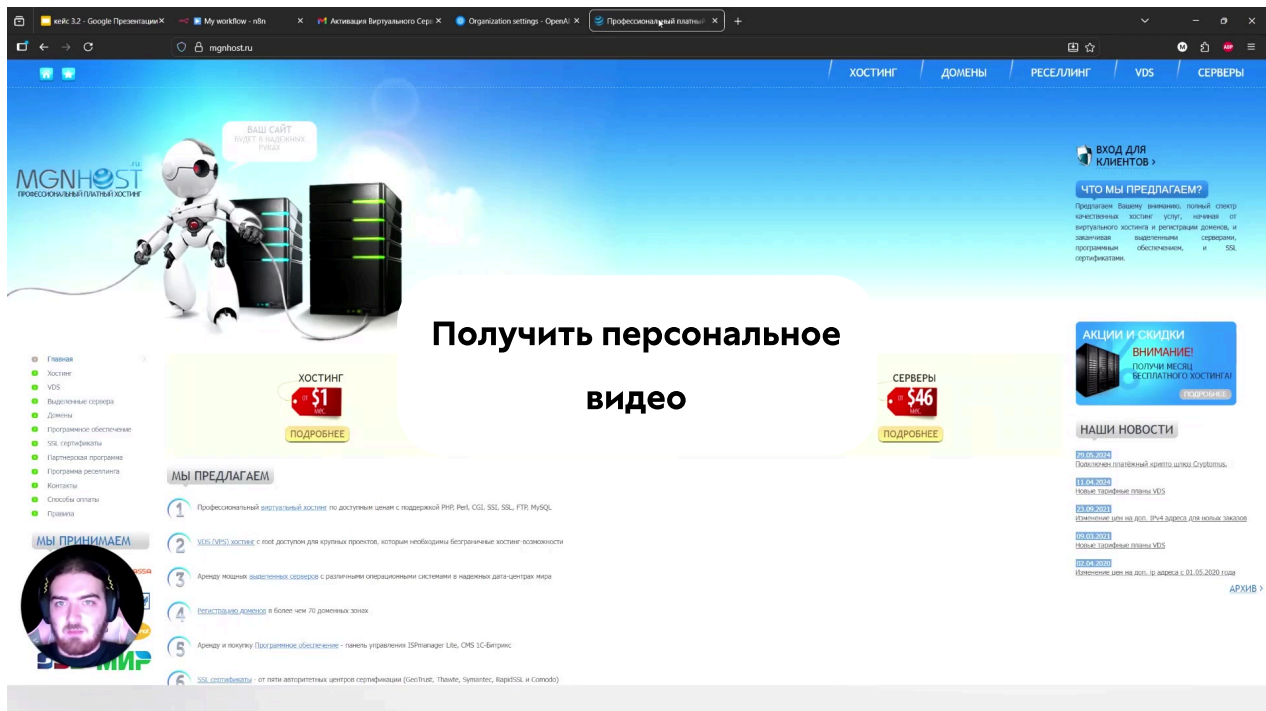
ⓘ Enterprise plan users can pull in credentials from external vaults. [More info](#)

⚠ Использование случайной строки из букв и цифр значительно повышает безопасность.

7. Сохраняем изменения → запускаем запрос → получаем статус 200 — запрос прошел успешно.
8. Переходим в Executions → проверяем выполнение. Если выполнить запрос без ключа — выполнение не отображается, доступ блокируется.

Система принимает запросы только при наличии корректного Authorization header , блокирует неавторизованные обращения, корректно выполняет генерацию поста и изображения. Защита настроена, проект готов к следующему этапу — развертыванию на сервере.

4. Запуск системы на сервере



► Подключение к серверу

5. Результаты урока

Итоги урока

- 1.** Повторили n8n, как рабочий инструмент для автоматизации задач
- 2.** Повторили как мы можем делать запросы к телеграм через API
- 3.** Дописали небольшой n8n скрипт для хостинга в телеграм используя orepai







► Итоги урока

Действия

Дополнительные материалы

- ▼ Архив проекта эксперта: код запросов, Docker команды, тексты промптов, n8n скрипт 

PErr10.5-10.6.zip

(<https://fs.getcourse.ru/fileservice/file/download/a/256825/sc/323/h/575dc0cad10c160eaafd762ac6692f3d.zip>)

- ▼ Руководство для Docker 

Руководство эксперта по разворачиванию Flask приложения с Docker (https://github.com/MrGAN12009/docker-help/blob/master/DOCKER_DEPLOYMENT_GUIDE.md)

- ▼ Код для запросов 

```
import requests #library for API requests
#Request headers/security
headers = {
    "Content-Type": "application/json",
    "Authorization": "Bearer 4d9e88d250ec15cfd2245906ee4e14ca8fc24e8
}
#Request to n8n for generation
r = requests.post("http://213.139.229.183:5678/webhook/generate-post
#Output status code from n8n:
#200 - success, everything went as expected
#404 - wrong url
#403 - problem with headers, no access
#Any other error code - https://developer.mozilla.org/en-US/docs/Web
#We search for our error number and see what went wrong
print(r.status_code)
```

- ▼ Ссылки на программы и сайты 

<https://mgnhost.ru/> (<https://mgnhost.ru/>) — **MGNHost**, VPS сервер для проектов на OpenAI.

Домашнее задание

Собери и разверни собственную AI-систему автопостинга в Telegram на базе n8n.

Часть 1. Сборка workflow

1. Создай workflow со следующей логикой:

Webhook → генерация текста → генерация промпта для изображения → генерация изображения → отправка в Telegram.

2. Настрой:

- генерацию текста поста (с ограничением токенов);
- отдельную ноду для генерации промпта изображения;
- генерацию изображения;
- от отправку фото в Telegram через sendPhoto (с caption);
- корректную передачу binary-данных.

Часть 2. Защита системы

1. Настрой авторизацию через Header Auth в Webhook.

2. Добавь Authorization: Bearer <token> в запросе.

3. Проверь:

- что без токена возвращается ошибка 403;
- что с токеном возвращается статус 200.

Часть 3. Развертывание

1. Разверни систему на удалённом сервере через Docker.

2. Импортируй workflow через JSON-файл.

3. Подключи credentials заново и проверь работу production URL.

Что нужно сдать

Пришли на проверку:

1. Скриншот полной схемы workflow в n8n.
2. Скриншот успешного Execution.
3. Скриншот итогового поста в Telegram (текст + изображение).
4. Скриншот ответа со статусом 200.
5. Скриншот ошибки 403 без токена.
6. JSON-файл экспортированного workflow.

Критерии зачета

- ✓ Пост публикуется автоматически.
- ✓ Генерируется и текст, и изображение.
- ✓ Настроена защита через Authorization header.
- ✓ Система работает с удалённого сервера.

На следующем занятии

На следующем уроке ты разработаешь ИИ-ассистента для прескоринга кандидатов на вакансию.

Мы разберем:

- как автоматически анализировать резюме кандидатов;
- как формировать критерии оценки (hard / soft skills, релевантность опыту);
- как выстраивать логику скоринга и выставления баллов;
- как генерировать структурированный отчет по каждому кандидату;
- как превратить это в прикладной HR-инструмент, который экономит часы ручной работы.

В итоге у тебя будет рабочий прототип AI-системы для первичной оценки кандидатов — еще один сильный кейс в портфолио 🚀

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PErr10.6 Генератор постов для блога в Telegram. Часть 2 (new)

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

Собери и разверни собственную AI-систему автопостинга в Telegram на базе **n8n**.

ЧАСТЬ 1. СБОРКА WORKFLOW

1. Создай workflow со следующей логикой:

Webhook → генерация текста → генерация промпта для изображения → генерация изображения → отправка в Telegram.

2. Настрой:

- ^{Действия} генерацию текста поста (с ограничением токенов);

- отдельную ноду для генерации промпта изображения;
- генерацию изображения;
- отправку фото в Telegram через `sendPhoto` (с `caption`);
- корректную передачу `binary`-данных.

ЧАСТЬ 2. ЗАЩИТА СИСТЕМЫ

1. Настрой авторизацию через `Header Auth` в `Webhook`.
2. Добавь `Authorization: Bearer <token>` в запросе.
3. Проверь:
 - что без токена возвращается ошибка `403`;
 - что с токеном возвращается статус `200`.

ЧАСТЬ 3. РАЗВЕРТЫВАНИЕ

1. Разверни систему на удалённом сервере через `Docker`.
2. Импортируй `workflow` через `JSON`-файл.
3. Подключи `credentials` заново и проверь работу `production URL`.

ЧТО НУЖНО СДАТЬ

Пришли на проверку:

1. Скриншот полной схемы `workflow` в `n8n`.
2. Скриншот успешного `Execution`.
3. Скриншот итогового поста в Telegram (текст + изображение).
4. Скриншот ответа со статусом `200`.
5. Скриншот ошибки `403` без токена.
6. `JSON`-файл экспортированного `workflow`.

КРИТЕРИИ ЗАЧЕТА

- ✓ Публикуется автоматически.

- ✓ Генерируется и текст, и изображение.
- ✓ Настроена защита через Authorization header.
- ✓ Система работает с удалённого сервера.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=341528145>)

Следующий урок

Действия